

DOKUMENTACJA PROJEKTU

Przewidywanie jakości czerwonego wina

System uczenia maszynowego z pipeline'em MLOps

Zespół	s27492, s26681, s27100
Problem ML	klasyfikacja binarna jakości wina
Zbiór danych	Wine Quality - Red (UCI), 1599 próbek
Repozytorium	GitHub (kod) + DVC (dane)

Spis treści

- 1** Wprowadzenie
- 2** Opis problemu
- 3** Opis danych
- 4** Architektura systemu
- 5** Pipeline ML (Kedro)
- 6** Eksperymenty i udoskonalanie modelu
- 7** Pipeline produkcyjny i serwowanie
- 8** Monitoring i detekcja dryftu
- 9** MLOps - wersjonowanie (Opcja A)
- 10** Sposób uruchomienia
- 11** Testy
- 12** Struktura repozytorium
- 13** Stos technologiczny

1. Wprowadzenie

Projekt realizuje kompletny proces uczenia maszynowego - od eksploracji danych, przez budowę i strojenie modelu, po wdrożenie modelu jako usługi sieciowej z monitoringiem. Celem jest przewidywanie jakości czerwonego wina na podstawie jego właściwości fizykochemicznych, bez potrzeby kosztownej oceny eksperckiej.

Dokumentacja opisuje problem, dane, architekturę systemu oraz sposób uruchomienia poszczególnych komponentów. Projekt zorganizowany jest wokół trzech równych pakietów pracy: (1) dane i model bazowy, (2) pipeline Kedro z eksperymentami i rejestrem modeli, (3) feature store, serwowanie, monitoring i konteneryzacja.

2. Opis problemu

Oryginalny atrybut quality to ocena ekspercka w skali 0-10 (w danych występują wartości 3-8). Ze względu na czytelność metryk i silne niezbalansowanie klas w ujęciu wieloklasowym, problem ujęto jako klasyfikację binarną:

Klasa	Definicja	Etykieta
Wino dobre	quality ≥ 6	1
Wino słabe	quality < 6	0

Metryki ewaluacji: accuracy, precision, recall, F1-score, ROC-AUC oraz macierz pomyłek. Jako wiodące traktowane są F1 i ROC-AUC, ponieważ lepiej oddają jakość klasyfikacji przy lekkim niezbalansowaniu klas.

3. Opis danych

3.1. Źródło i charakterystyka

- Zbiór: Wine Quality - Red (P. Cortez i in., UCI Machine Learning Repository).
- Plik: data/01_raw/winequality-red.csv, wersjonowany przez DVC.
- Liczba próbek: 1599; liczba cech: 11 (numeryczne, ciągłe) + kolumna celu quality.
- Braki danych: 0. Duplikaty wierszy: 240 (usuwane w preprocessingu).

3.2. Balans klas (po binaryzacji)

Klasa	Liczność	Udział
1 - dobre (quality ≥ 6)	855	53,5%
0 - słabe (quality < 6)	744	46,5%

Zbiór jest lekko niezbalansowany - akceptowalny bez specjalnych technik; w razie potrzeby można zastosować class_weight lub SMOTE.

3.3. Cechy (zmienne objaśniające)

Cecha	Opis
fixed acidity	kwasowość stała (kwas winowy)
volatile acidity	kwasowość lotna (kwas octowy) - wysoka pogarsza smak

citric acid	kwasek cytrynowy - świeżość
residual sugar	cukier resztkowy
chlorides	zawartość soli
free sulfur dioxide	wolny SO ₂ - ochrona przed utlenianiem
total sulfur dioxide	całkowity SO ₂
density	gęstość
pH	kwasowość w skali pH
sulphates	siarczany - dodatek konserwujący
alcohol	zawartość alkoholu (silnie skorelowana z jakością)

3.4. Wnioski z EDA i model bazowy

- Najsilniejsze korelacje z jakością: alcohol (+), sulphates (+), volatile acidity (-).
- Usunięto 240 duplikatów; pozostałe cechy bez braków.
- Zastosowano StandardScaler (cechy w różnych skalach) i stratyfikowany podział 80/20.

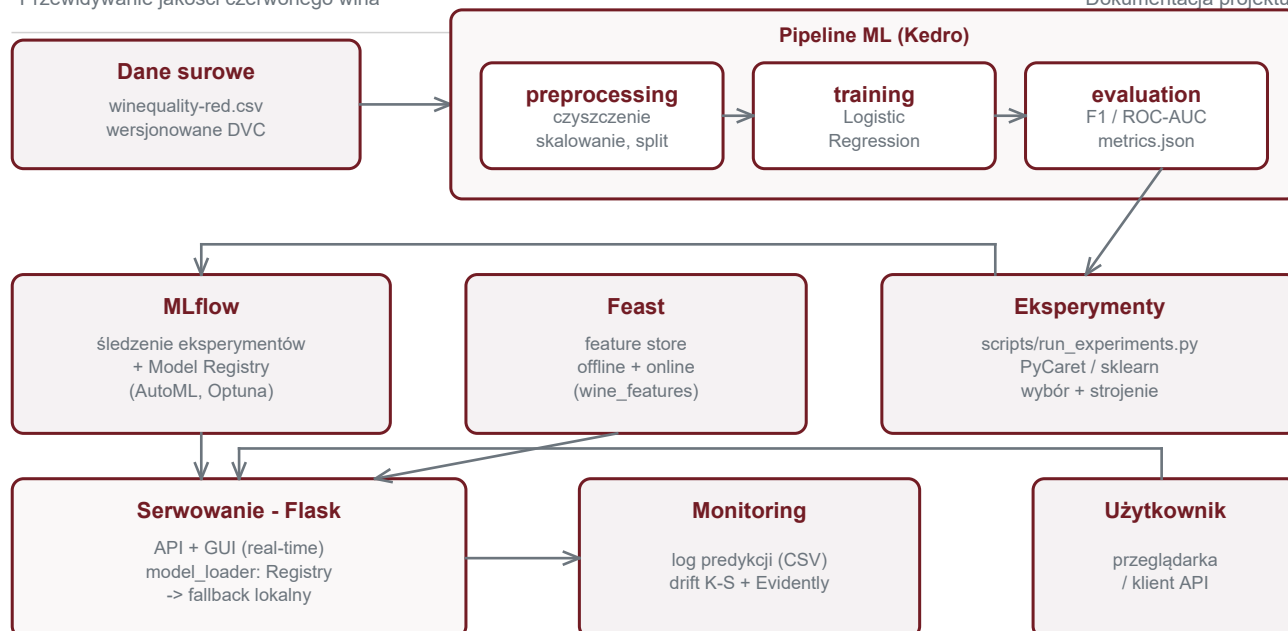
Model bazowy (Logistic Regression) na zbiorze testowym:

Metryka	Wartość
accuracy	0,735
precision	0,761
recall	0,729
F1	0,745
ROC-AUC	0,812

Artefakt bazowy (scaler + model + lista cech + metryki) stanowi punkt odniesienia, który etap udoskonalania (AutoML + strojenie) ma poprawić.

4. Architektura systemu

System składa się z czterech warstw: warstwy danych (DVC), pipeline'u treningowego (Kedro + MLflow), repozytoriów MLOps (DVC, Feast, MLflow Model Registry) oraz warstwy serwowania (Flask API/GUI z monitoringiem). Poniższy schemat przedstawia przepływ danych i zależności między komponentami.



4.1. Komponenty

- Warstwa danych - surowy CSV w data/01_raw/, warstwy pośrednie Kedro (02_intermediate, 05_model_input, 06_models, 08_reporting); dane wersjonowane przez DVC.
- Pipeline ML (Kedro) - trzy pipeline'y: data_preprocessing, data_science (training), evaluation; spięte w pipeline_registry.py jako pipeline domyślny.
- Eksperymenty - scripts/run_experiments.py: AutoML (PyCaret lub fallback scikit-learn), strojenie Optuna, logowanie do MLflow i rejestracja modelu.
- MLflow - śledzenie eksperymentów (sqlite:///mlflow.db) oraz Model Registry (model wine_quality, stage Production).
- Feast - feature store (offline + online) z encją wine i widokiem wine_features.
- Serwowanie (Flask) - API + GUI, inferencja w trybie real-time, monitoring predykcji i detekcja dryftu; konteneryzacja przez Docker.

4.2. Przepływ danych

CSV (raw) -> [DVC] -> Kedro: preprocessing -> training -> evaluation. Równolegle skrypt eksperymentów trenuje i stroi modele, loguje przebiegi do MLflow i rejestruje najlepszy model w Model Registry. Warstwa serwowania ładuje model (najpierw z Registry, a w razie niedostępności z lokalnego artefaktu), udostępnia predykcje przez API/GUI, pobiera cechy z Feast po identyfikatorze oraz loguje predykcje na potrzeby detekcji dryftu.

5. Pipeline ML (Kedro)

Kod produkcyjny w src/winequality/ jest refaktoryzacją notebooka bazowego do modularnego pipeline'u Kedro. Konfiguracja danych znajduje się w conf/base/catalog.yml, a parametry w conf/base/parameters.yml.

5.1. Pipeline'y i węzły

Pipeline	Węzeł	Wejście -> Wyjście
data_preprocessing	clean_data	wine_raw -> wine_cleaned
data_preprocessing	split_and_scale_data	wine_cleaned -> X_train/X_test/y_train/y_test
data_science	train_baseline_model	X_train, y_train -> baseline_logreg

evaluation	evaluate_model	baseline_logreg, X_test, y_test -> metrics
------------	----------------	--

- clean_data - usuwa duplikaty i braki, tworzy target binarny (quality >= threshold) i usuwa kolumnę quality.
- split_and_scale_data - podział stratyfikowany train/test i skalowanie StandardScaler.
- train_baseline_model - LogisticRegression (C=1.0, max_iter=1000, solver=lbfgs); loguje parametry, metryki treningowe i artefakt do MLflow (z bezpiecznym fallbackiem).
- evaluate_model - liczy accuracy, F1 i ROC-AUC, zapisuje metrics.json w data/08_reporting/.

5.2. Parametry

Parametr	Wartość	Znaczenie
target_threshold	6	próg binaryzacji celu (quality >= 6 -> 1)
random_state	42	ziarno losowości (powtarzalność)
test_size	0.2	udział zbioru testowego

5.3. Uruchomienie

```
kedro run # cały pipeline domyślny
kedro run --pipeline=data_preprocessing
kedro run --pipeline=evaluation
```

6. Eksperymenty i udoskonalanie modelu

Skrypt scripts/run_experiments.py realizuje etap udoskonalania modelu: porównanie wielu modeli (AutoML), inżynierię i strojenie hiperparametrów oraz śledzenie i rejestrację wyników w MLflow.

6.1. AutoML - porównanie modeli

- Ścieżka główna: PyCaret compare_models (lr, rf, et, dt oraz xgboost/lightgbm gdy dostępne), sortowanie wg F1.
- Ścieżka awaryjna (gdy brak PyCaret): ręczne porównanie scikit-learn z 5-krotną walidacją krzyżową StratifiedKFold dla LogisticRegression, RandomForest, ExtraTrees, DecisionTree, GradientBoosting (+ XGBoost/LightGBM gdy dostępne).
- Wynik: tabela porównawcza zapisywana do data/08_reporting/model_comparison.csv; każdy model logowany jako osobny przebieg MLflow.

6.2. Strojenie hiperparametrów - Optuna

Najlepszy model z etapu AutoML jest strojony przez Optunę (sampler TPE, optymalizacja bayesowska, 30 prób), z funkcją celu = średnia F1 z walidacji krzyżowej. Nastrojony model jest ewaluowany na wydzielonym zbiorze testowym (F1, ROC-AUC).

6.3. MLflow - śledzenie i Model Registry

- Tracking URI: sqlite:///mlflow.db; eksperymenty wine_quality_experiments i wine_quality_baseline.
- Najlepszy nastrojony model jest logowany i rejestrowany pod nazwą wine_quality, a następnie promowany do stage Production (z archiwizacją poprzednich wersji; fallback do aliasu Production dla nowszych konfiguracji MLflow).
- Kopia najlepszego modelu zapisywana lokalnie do data/06_models/best_model.pkl (na potrzeby serwowania bez aktywnego Registry).

6.4. Uruchomienie

```
python scripts/run_experiments.py
```

7. Pipeline produkcyjny i serwowanie

Pakiet serving/ udostępnia model jako aplikację Flask (API + proste GUI). Inferencja działa w trybie real-time (pojedyncza predykcja na żądanie), co odpowiada interaktywnemu GUI i prostemu demo - przetwarzanie wsadowe nie jest tu potrzebne.

7.1. Endpointy API

Metoda	Ścieżka	Opis
GET	/	formularz GUI (11 cech) -> dobre/słabe + prawdopodobieństwo
POST	/predict	JSON {"features": {...}} lub lista 11 wartości -> predykcja
POST	/predict/by_id	pobiera cechy z Feast po wine_id, potem predykcja
GET	/health	status usługi + źródło modelu
GET	/monitoring/drift	raport dryftu (K-S per cecha + opcjonalnie Evidently)

7.2. Ładowanie modelu

Moduł model_loader.py realizuje strategię: najpierw próba pobrania modelu z MLflow Model Registry (models:/wine_quality/Production) - aktywna gdy ustawiona jest zmienna MLFLOW_TRACKING_URI; w razie niepowodzenia następuje fallback do lokalnego artefaktu (best_model.pkl, a następnie baseline_logreg.pkl). Dla surowego klasyfikatora scaler jest odtwarzany z danych treningowych, a dla sklearn Pipeline stosowany jest scaler pass-through.

7.3. Przykład zapytania

```
curl -X POST localhost:5000/predict -H 'Content-Type: application/json' \
  -d '{"features": {"fixed acidity":7.4,"volatile acidity":0.7,
    "citric acid":0.0,"residual sugar":1.9,"chlorides":0.076,
    "free sulfur dioxide":11,"total sulfur dioxide":34,
    "density":0.9978,"pH":3.51,"sulphates":0.56,"alcohol":9.4}}'
```

8. Monitoring i detekcja dryftu

- Logowanie predykcji - każda predykcja (znacznik czasu + 11 cech + wynik) zapisywana jest do serving/logs/predictions.csv.
- Detekcja dryftu - endpoint /monitoring/drift porównuje rozkład zalogowanych predykcji z rozkładem treningowym (reference_stats.parquet) testem Kołmogorowa-Smirnowa per cecha; p-value < 0,05 oznacza dryft cech.
- Raport - wynik zapisywany do serving/logs/drift_report.json (udział cech z dryftem, flaga dataset_drift); opcjonalnie raport HTML Evidently (DataDriftPreset), jeśli biblioteka jest dostępna.

9. MLOps - wersjonowanie (Opcja A)

Wybrano Opcję A wymagań (Repozytoria): pełny łańcuch wersjonowania danych, cech i modeli.

Repozytorium	Narzędzie	Zakres
Wersjonowanie danych	DVC	surowy CSV poza Gitem; w Git tylko wskaźniki *.dvc i .dvc/config
Feature Store	Feast	encja wine (wine_id), widok wine_features (11 cech), offline + online
Repozytorium modeli	MLflow Model Registry	model wine_quality, wersjonowanie i stage Production

Workflow zespołu: git pull (kod + wskaźniki .dvc), a następnie dvc pull (odtworzenie danych z remote).
Feast: feast apply oraz materializacja przed użyciem online store.

10. Sposób uruchomienia

10.1. Środowisko

Wymagana Miniconda/Anaconda. Wspólne środowisko conda (Python 3.11):

```
conda env create -f environment.yml
conda activate wine
dvc pull          # po skonfigurowaniu remote DVC
```

10.2. Notebook bazowy, pipeline i eksperymenty

```
jupyter lab notebooks/01_baseline_eda.ipynb # EDA + model bazowy
kedro run                                     # pipeline Kedro
python scripts/run_experiments.py           # AutoML + Optuna + MLflow
```

10.3. Serwowanie - lokalnie

```
pip install -r serving/requirements.txt
python serving/bootstrap_data.py          # data/01_raw/winequality-red.csv
python serving/bootstrap_model.py         # data/06_models/baseline_logreg.pkl
cd feature_repo && python generate_features.py && feast apply && cd ..
gunicorn serving.app:app -b 0.0.0.0:5000 # lub: flask --app serving/app run
```

GUI dostępne pod adresem <http://localhost:5000>.

10.4. Serwowanie - Docker

```
docker compose -f docker/docker-compose.yml up --build
```

Uwaga: gdy MLflow Model Registry jest aktywny, ustaw zmienną MLFLOW_TRACKING_URI - serwowanie pobierze wówczas model models:/wine_quality/Production. Bez niej używany jest lokalny artefakt (best_model.pkl lub baseline_logreg.pkl).

11. Testy

Testy automatyczne (pytest) znajdują się w katalogu tests/ i pokrywają kluczowe moduły:

Plik	Zakres
test_data.py	spójność i kształt danych wejściowych
test_preprocessing.py	czyszczenie, binaryzacja celu, podział i skalowanie
test_serving.py	ładowanie modelu i endpointy API
test_monitoring.py	logowanie predykcji i obliczanie dryftu

```
pytest -q
```

12. Struktura repozytorium

data/	warstwy danych Kedro (wersjonowane DVC)
01_raw/	surowe dane: winequality-red.csv
06_models/	artefakty modeli (.pkl)
notebooks/	01_baseline_eda.ipynb (EDA + model bazowy)
src/winequality/	kod produkcyjny / pipeline Kedro
pipelines/	data_preprocessing, data_science, evaluation
conf/base/	catalog.yml, parameters.yml
scripts/	run_experiments.py (AutoML + Optuna + MLflow)
feature_repo/	definicje i generowanie cech Feast
serving/	Flask API + GUI + monitoring
docker/	Dockerfile, docker-compose.yml, entrypoint.sh
tests/	testy pytest
docs/	dokumentacja, plan, opis problemu i danych
environment.yml	definicja środowiska conda

13. Stos technologiczny

- Język i środowisko: Python 3.11, conda.
- Dane i ML: pandas, scikit-learn, Jupyter.
- Pipeline i eksperymenty: Kedro, MLflow, PyCaret, Optuna.
- MLOps: DVC (dane), Feast (feature store), MLflow Model Registry (modele).
- Serwowanie i wdrożenie: Flask, gunicorn, Docker / docker-compose.
- Monitoring: scipy (test K-S), Evidently (opcjonalny raport HTML).
- Testy: pytest.